Milestone 2 Report – SmartLink Software Design Project: SmartLink – Professional Networking Platform

1. Team Members

Name	ID
Omar Khaled Hussein	22010962
Ayman Ibrahim Mohamed Kotb	22010656
Ali El-Deen Maher Abd El-Moniem	22010934
Ali Mostafa Mohamed	22010939
Yaseen Asaad Ahmed Farid	22011349
Farouk Ashraf Farouk	22011012

---

2. Milestone Objective

Establish SmartLink as an interactive professional networking platform by implementing core social features that enable users to connect, collaborate, and engage with each other professionally. This milestone focuses on transforming the platform from individual profiles to an interconnected professional community.

---

3. Scope & Implemented Features

Core Scope:

- Connection Management System for building professional networks

- Advanced Search Functionality for discovering opportunities
- Recruiter Tools for talent acquisition
- Administrative Content Moderation
- Basic Notification System

Implemented Features:

3.1 Connection Management System (Req. 2.6)

- **Connection Requests:** Send, receive, accept, and decline connection requests
- **Network Visualization:** View current connections, pending requests, and mutual connections
- **Profile Access:** View detailed profiles of connections
- **Network Statistics:** Track connection count and network growth

3.2 Search Functionality (Req. 2.7)

- **Multi-criteria Search:** Search users by name, skills, location, or job title
- **Job Search:** Find job postings by title, company, or required skills
- **Advanced Filters:**
  - Skills multi-select filter
  - Experience level filtering
  - Location-based search
  - Industry/company size filters
- **Auto-suggestions:** Real-time search suggestions as users type

3.3 Recruiter Features

- **Job Posting Management (Req. 3.1):**
  - Create, edit, and delete job postings
  - Draft saving capability
  - Comprehensive job form with validation
  - Salary range and application deadline management
- **Candidate Search (Req. 3.2):**
  - Advanced candidate filtering by skills and experience
  - Skills match percentage calculation
  - Candidate profile quick view
  - Export search results capability

3.4 Content Moderation (Admin Dashboard) (Req. 4.2)

- **Moderation Queue:** View and manage reported content

- **User Management:** Ban repeat offenders
- **Audit Logging:** Track moderation actions

---

4. Functional Requirements

4.1 Connection Management Requirements:

- Users can send connection requests to other professionals
- Users can accept or decline incoming connection requests
- Users can view their current connections and pending requests
- Connection removal functionality for managing networks
- Mutual connections display for building trust
- Network growth analytics and statistics

4.2 Search System Requirements:

- Real-time search with auto-suggestions
- Multi-field search (name, skills, company, job title)
- Advanced filtering system with clear UI
- Paginated search results for performance
- Search result relevance scoring
- Save search queries for future use

4.3 Moderation System Requirements:

- Secure admin authentication and authorization
- Content reporting mechanism for users
- Moderation action logging and audit trails
- Email notifications for warned users
- Ban functionality with appropriate checks
- Dashboard analytics for moderation activities

4.4 Job Posting Management

1: create Job

Actor: Company User

Input:

1. Company ID

2. Job Title
3. Job Description
4. Experience Level
5. Job Type
6. Location Type
7. Job Location
8. salary range
9. Application Deadline

Process:

1. Validate company ID exists
2. Verify user type is "COMPANY"
3. Create job entity with provided information
4. Save to database

Output: JobResponse containing job details

Postconditions:

1. New job posting is stored in database
2. Jobs are visible in current jobs list

2: Update Job Posting

Actor: Company User

Input:

- Job id
- Updated job details

Process:

1. Retrieve existing job by ID
2. Update all modifiable fields
3. Save changes to database

Output: Updated JobResponse

Postconditions:

- Job posting reflects new information

3: Delete Job Posting

Actor: Company User

Input: Job ID

Process:

1. Remove job from database

Output: delete job from database

Postconditions:

- Job post is permanently removed
- Associated data handling by cascading

4: View Current Job Postings

Actor: Company User

Input:

- Company ID
- Page number
- Page size

Process:

1. Query jobs where deadline > current time
2. Sort by creation date (ascending)
3. Apply pagination

Output: list of Jobs

5: View Ended Job Postings

Actor: Company User

Input:

- Company ID
- Page number
- Page

Process:

1. Query jobs where deadline < current time
2. Sort by deadline (ascending)
3. Apply pagination

Output: Paginated list of Jobs

4.6 Search and Apply for a job:

1: search for Job

Actor: Company User

Input:

1. Job Title
2. Job Description
3. Experience Level
4. Job Type
5. Location Type
6. Job Location
7. salary range
8. Application Deadline

Process:

1. Filter data base by incoming data

Output: jobs user asker for

2: Apply

Actor: Company User

Input:

1. Job data
2. CV
3. Full name
4. Email
5. cover letter

Process:

1. validate user data and job data
2. validate user didn't apply before
3. post application

Output: jobs user asker for

4.7 Comment Management System

1: Add Comment

Actor: User

Input:

1. User ID
2. Post ID
3. Comment Text
4. Attachment URL (optional)
5. Attachment Type (optional)

Process:

1. Fetch user entity from database by user ID
2. Fetch post entity from database by post ID
3. If attachment URL provided, create Attachment entity with URL and type
4. Build Comment entity with user, post, content, attachment.
5. Save comment to database
6. Return saved comment ID

Output: Comment ID (Long)

2: Get Comments by Post

Actor: User

Input:

1. Post ID
2. Page Number
3. Page Size

Process:

1. Create pageable object with page number and size
2. Query database for comments by post ID with pagination
3. Convert each Comment entity to CommentDTO
4. Map attachment data if present
5. Return list of CommentDTOs

Output: List of CommentDTO (paginated)

4.8 Post Attachment Management System

1: Find All Post Attachments

Actor: System/Admin

Input: None

Process:

1. Query database for all PostAttachment records
2. Return complete list

Output: List of PostAttachment entities

2: Find Posts by Attachment ID

Actor: System

Input:

1. Attachment ID

Process:

1. Query database for all posts associated with the attachment ID
2. Return list of post IDs

Output: List of Post IDs (Long)

3: Find Attachments by Post ID

Actor: System

Input:

1. Post ID

Process:

1. Query database for all attachments associated with the post ID
2. Return list of attachment IDs

Output: List of Attachment IDs (Long)

4: Delete Post Attachments

Actor: User/System

Input:

1. Post ID

Process:

1. Find all attachment relationships for the post
2. Delete all PostAttachment records for this post ID
3. Complete transaction

Output: Void (deletion successful)

5: Delete Specific Attachment from Post

Actor: User/System

Input:

1. Attachment ID
2. Post ID

Process:

1. Find specific PostAttachment record by both IDs
2. Delete the relationship record
3. Complete transaction

Output: Void (deletion successful)

6: Save Post Attachment

Actor: User

Input:

1. PostAttachment entity (containing Post ID and Attachment ID)

Process:

1. Validate post and attachment exist
2. Save PostAttachment relationship to database
3. Return saved entity with generated ID

Output: Saved PostAttachment entity

7: Update Post ID in Attachment Relationship

Actor: System

Input:

1. Post ID
2. Attachment ID

Process:

1. Find PostAttachment record by attachment ID
2. Update the post ID in the relationship
3. Save changes to database

Output: Void (update successful)

8: Update Attachment ID in Post Relationship

Actor: System

Input:

1. Attachment ID
2. Post ID

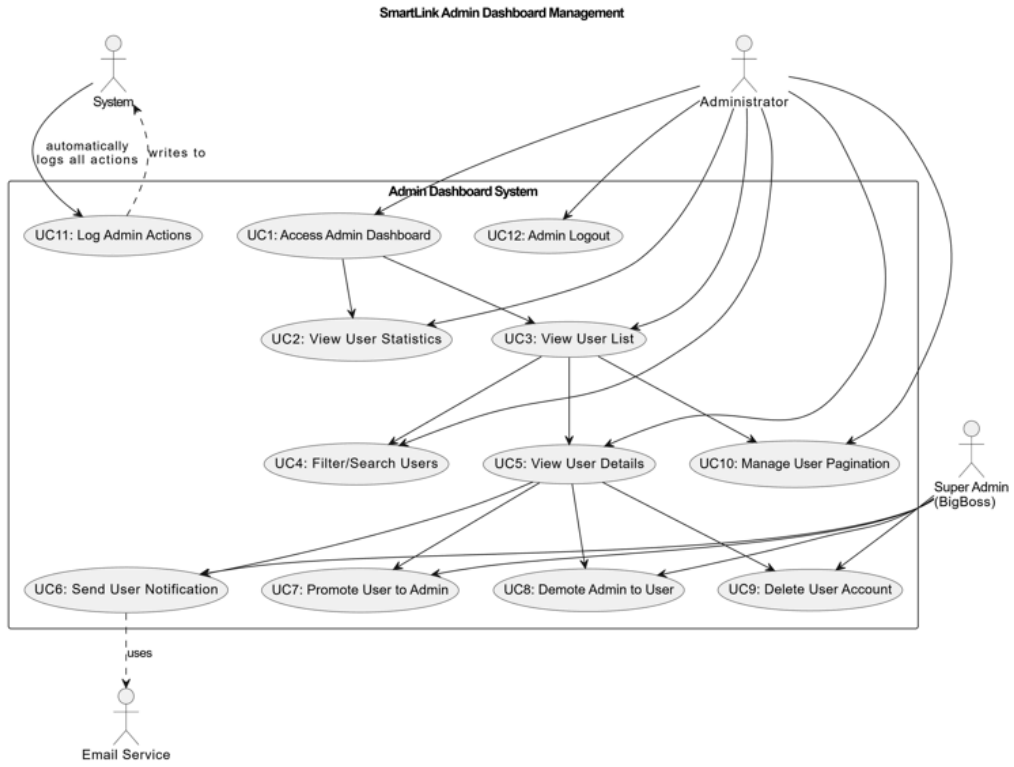
Process:

1. Find PostAttachment record by post ID
2. Update the attachment ID in the relationship
3. Save changes to database

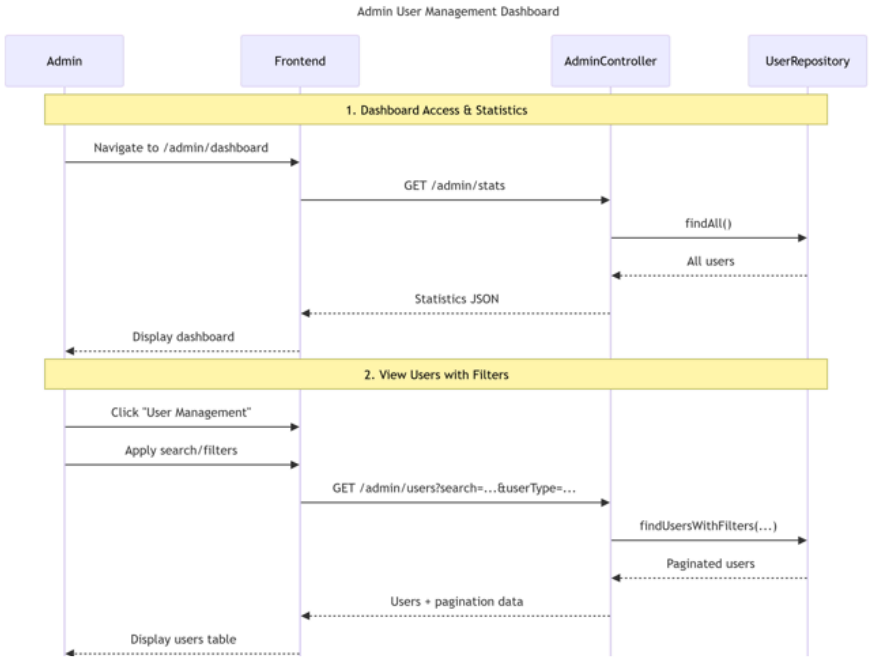
Output: Void (update successful)

5. Important Use Case and Sequence/Activity diagram Summary

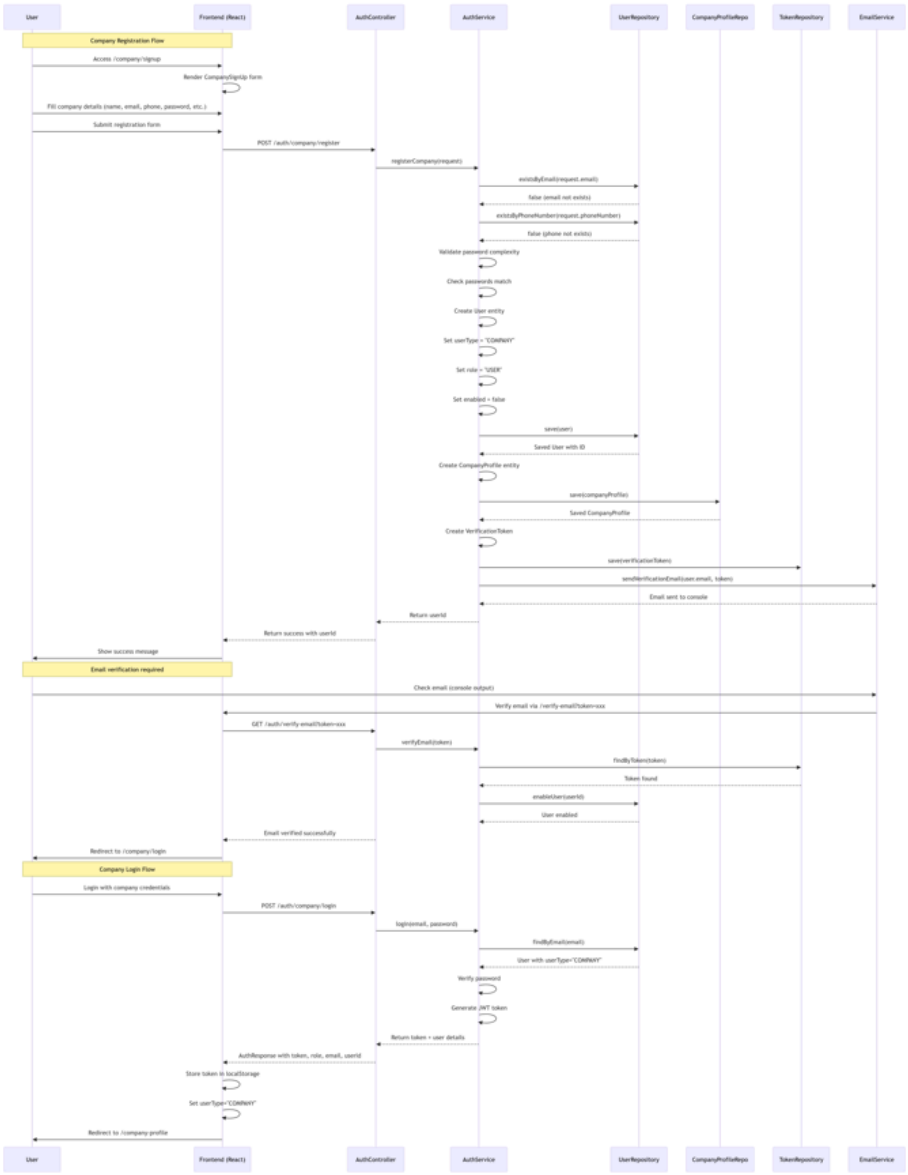
Use case for Admin Dashboard



Detailed Admin User management sequence diagram

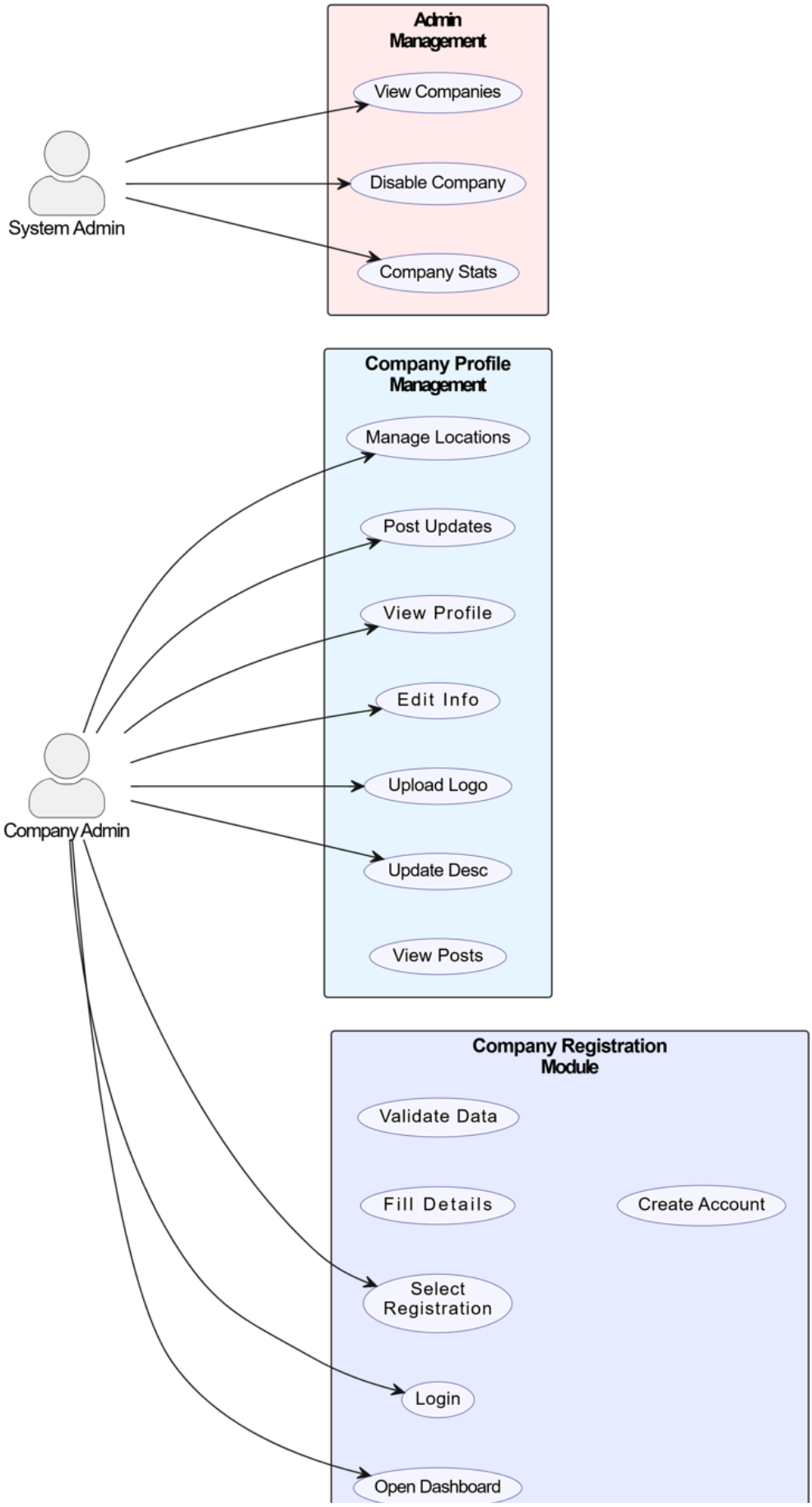


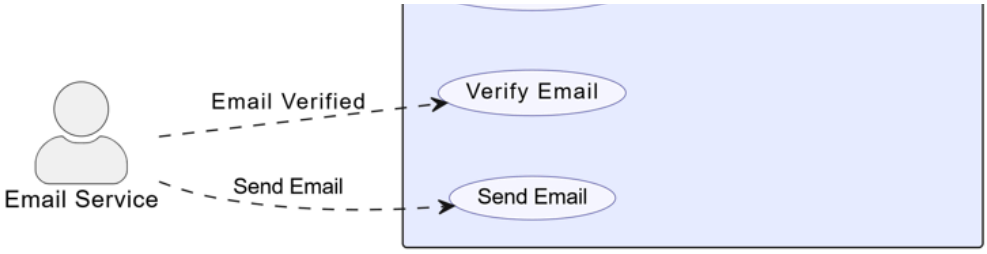
Detailed sequence diagram for Company registration



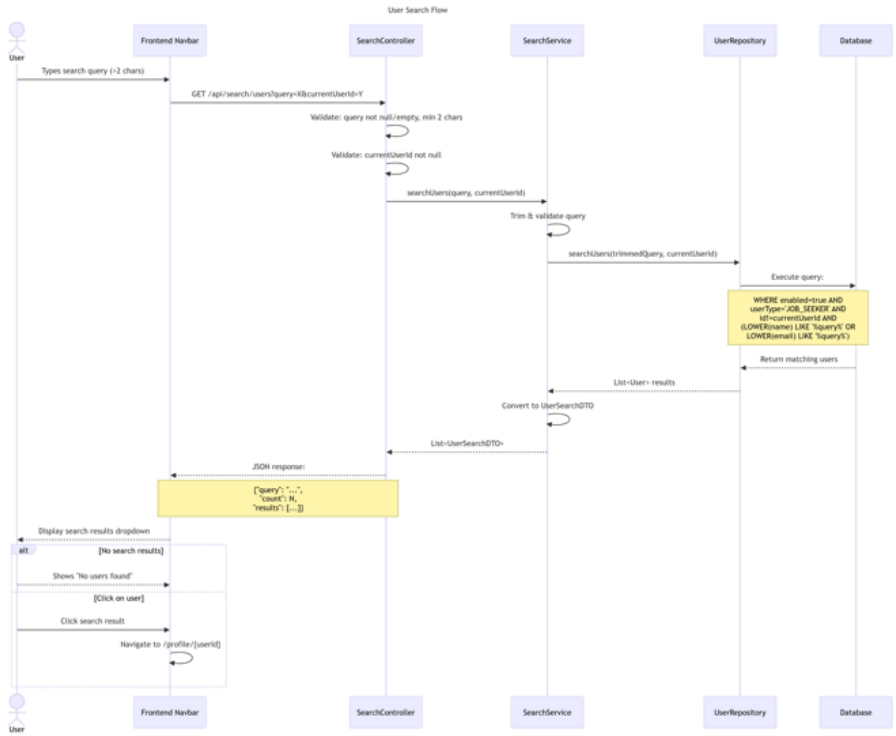
company registration use case

SmartLink Company Registration System



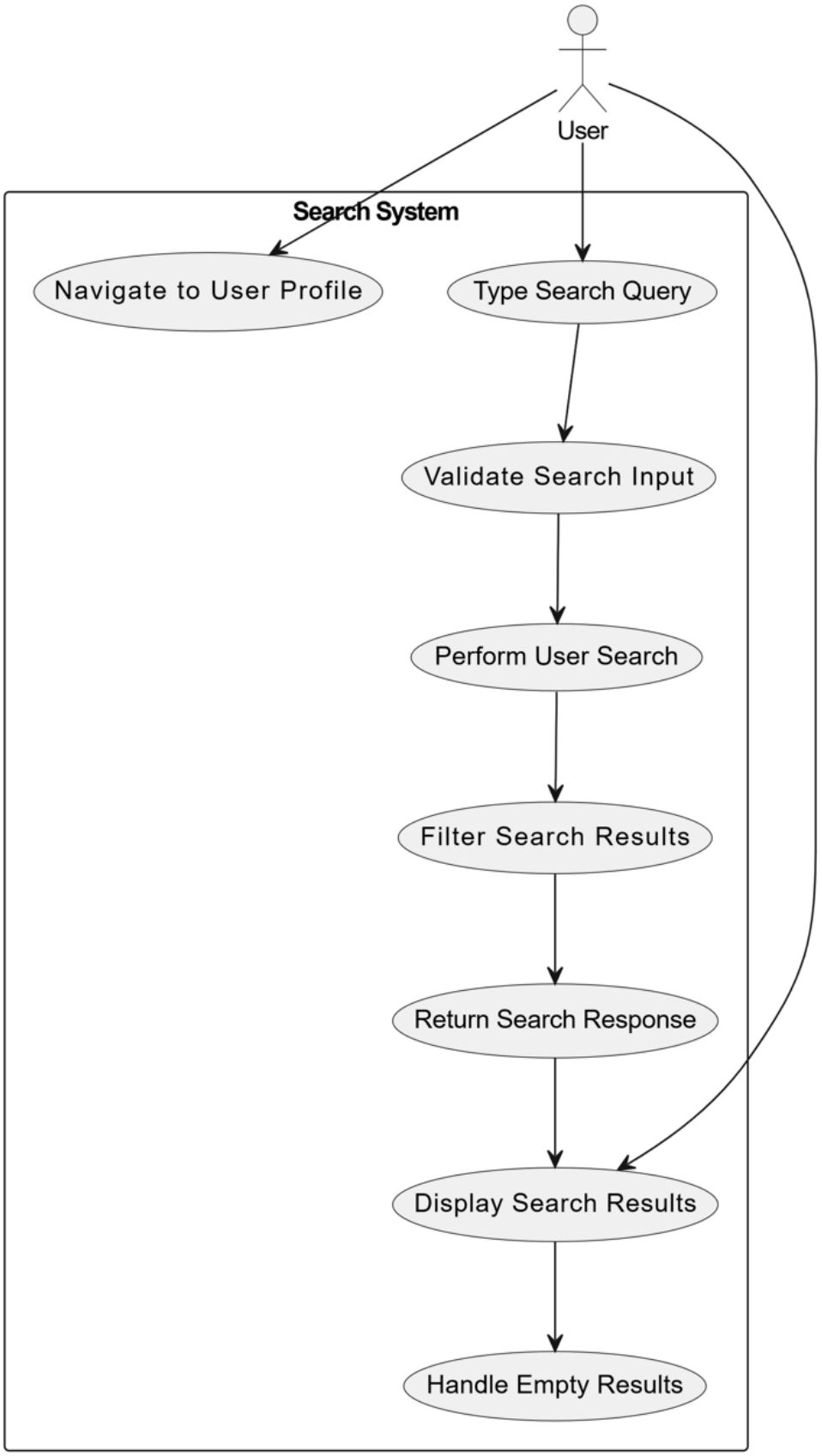


Sequence diagram for the user search functionality

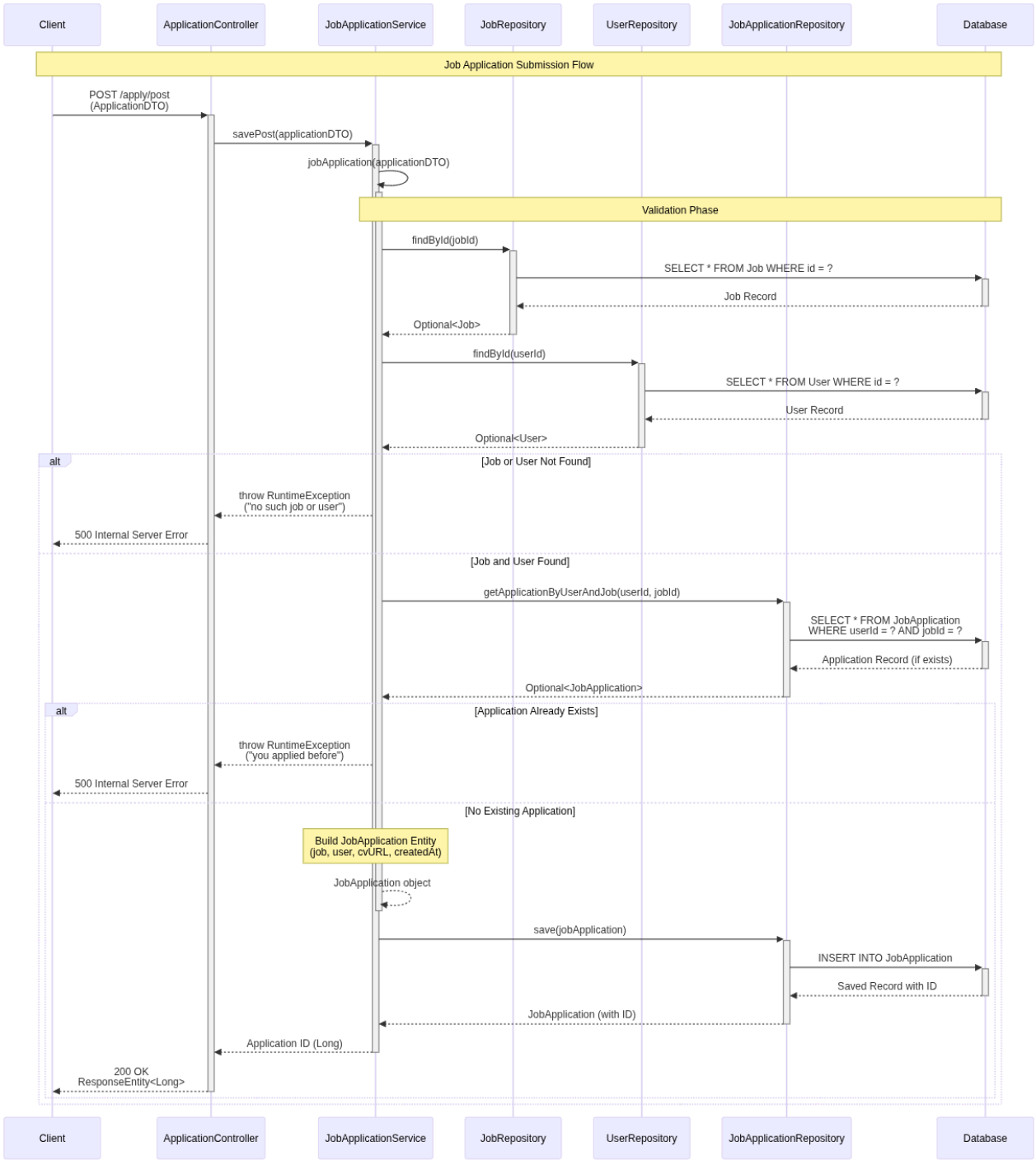


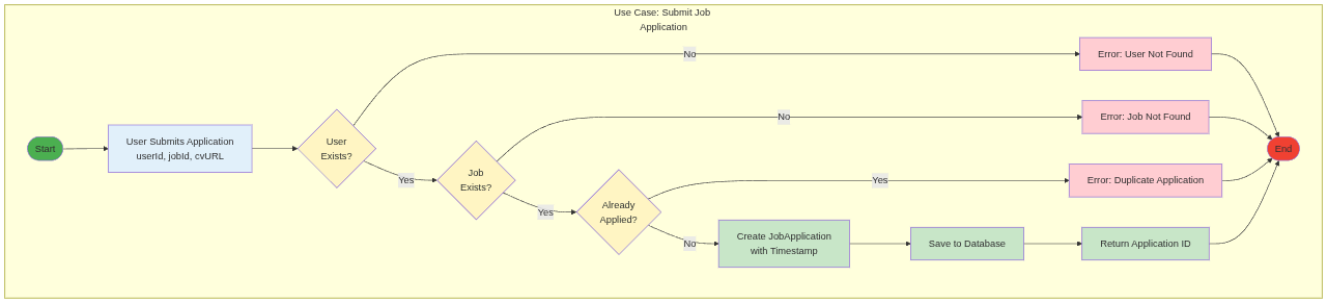
Use case diagram for the user search functionality

User Search - Use Case Diagram

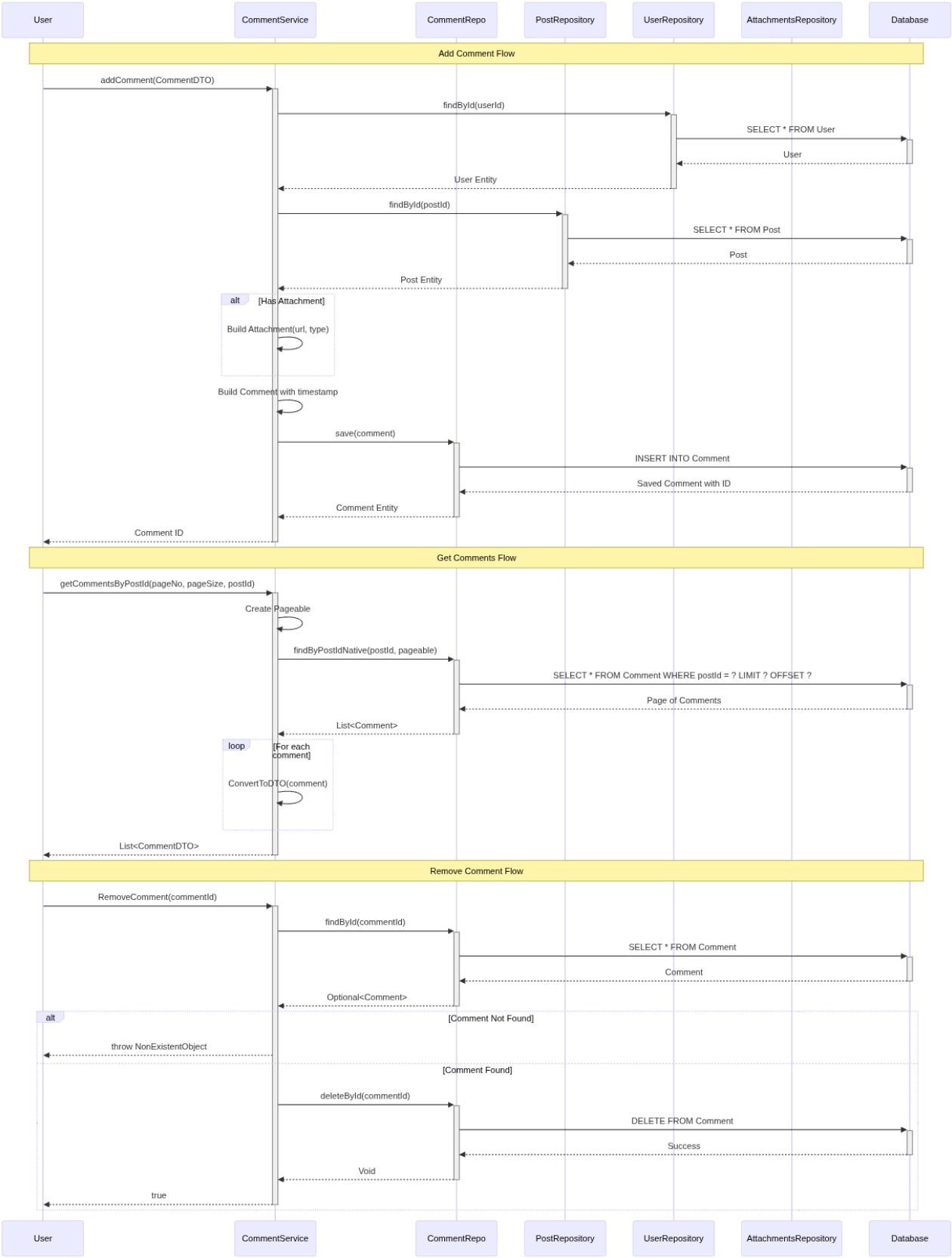


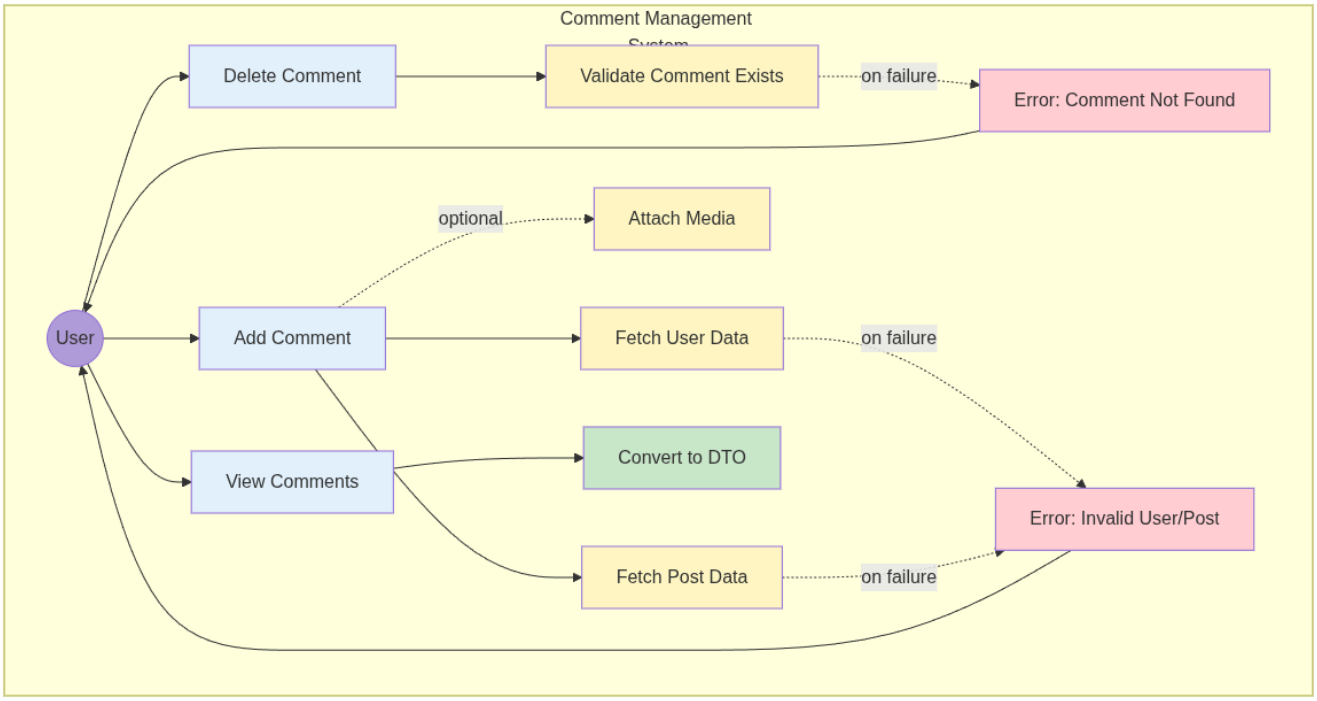
Apply for a job



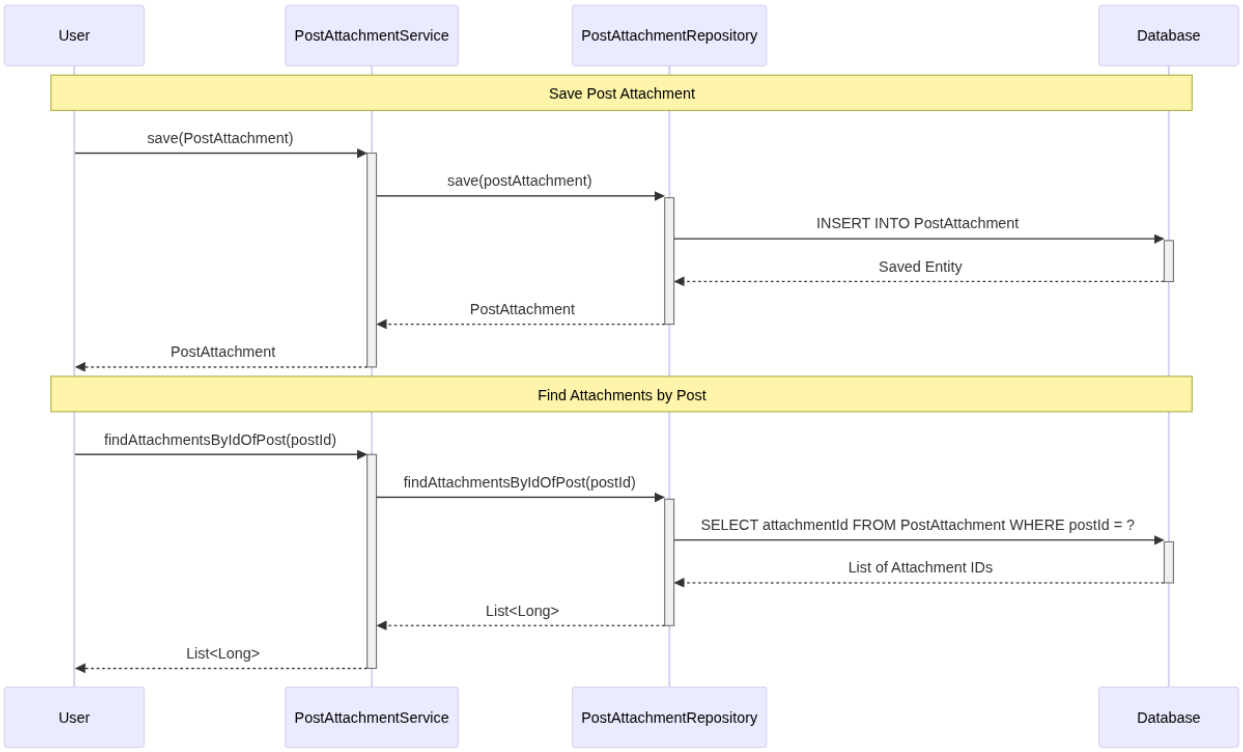


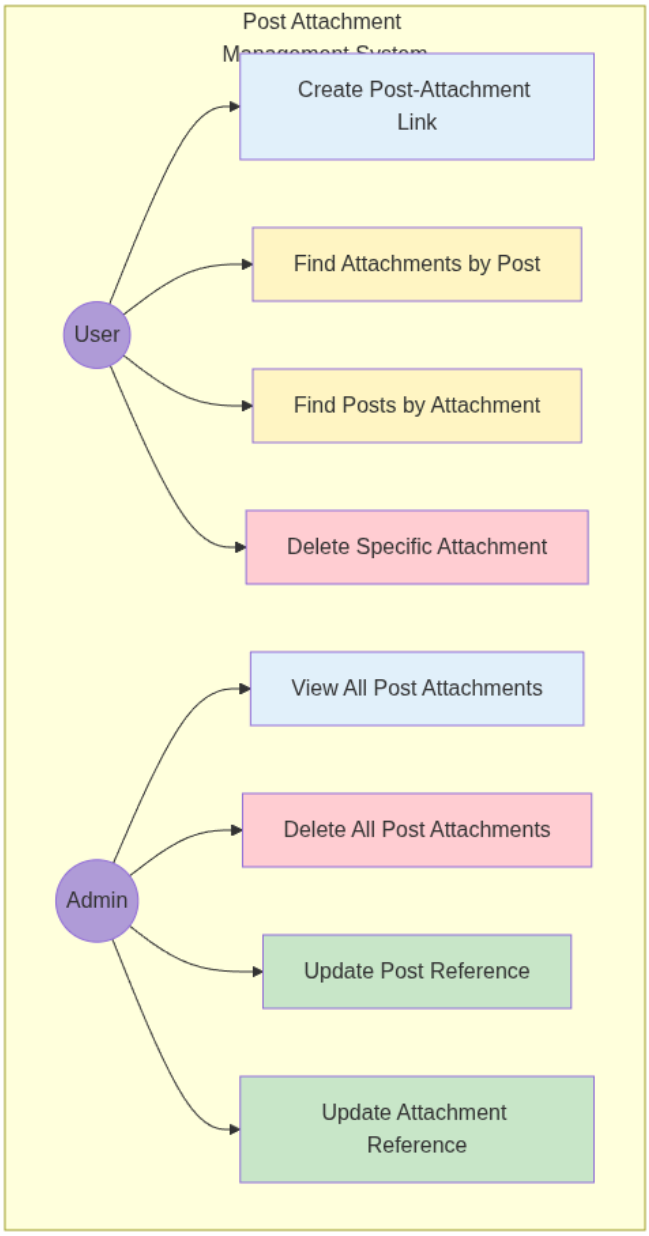
Comment management


User
CommentService
CommentRepo
PostRepository
UserRepository
AttachmentsRepository
Database
Add Comment Flow
addComment(CommentDTO)
findById(userId)
SELECT * FROM User
User
User Entity
findById(postId)
SELECT * FROM Post
Post
Post Entity
alt
[Has Attachment]
Build Attachment(url, type)
Build Comment with timestamp
save(comment)
INSERT INTO Comment
Saved Comment with ID
Comment Entity
Comment ID
Get Comments Flow
getCommentsByPostId(pageNo, pageSize, postId)
Create Pageable
findByPostIdNative(postId, pageable)
SELECT * FROM Comment WHERE postId = ? LIMIT ? OFFSET ?
Page of Comments
List<Comment>
loop
[For each comment]
ConvertToDTO(comment)
List<CommentDTO>
Remove Comment Flow
RemoveComment(commentId)
findById(commentId)
SELECT * FROM Comment
Comment
Optional<Comment>
alt
[Comment Not Found]
throw NonExistentObject
[Comment Found]
deleteById(commentId)
DELETE FROM Comment
Success
Void
true

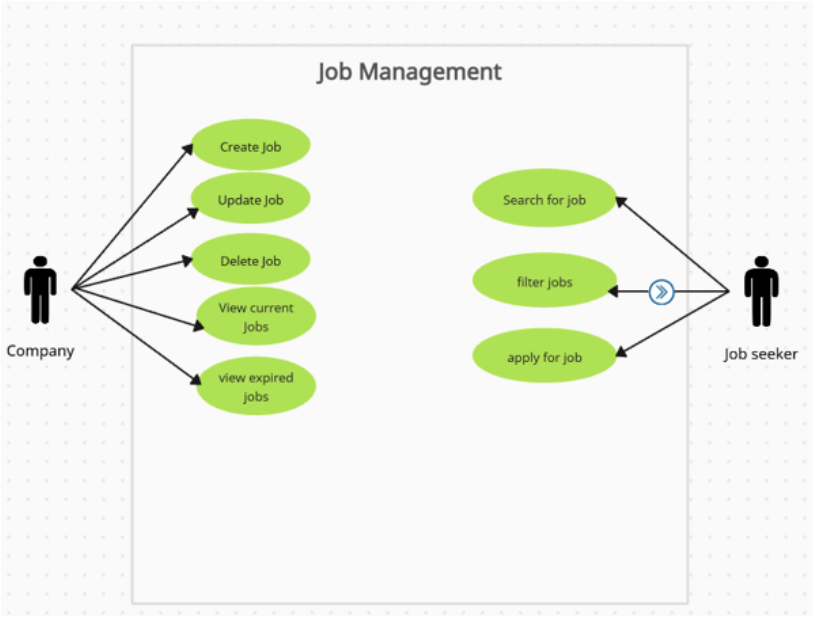


Post management



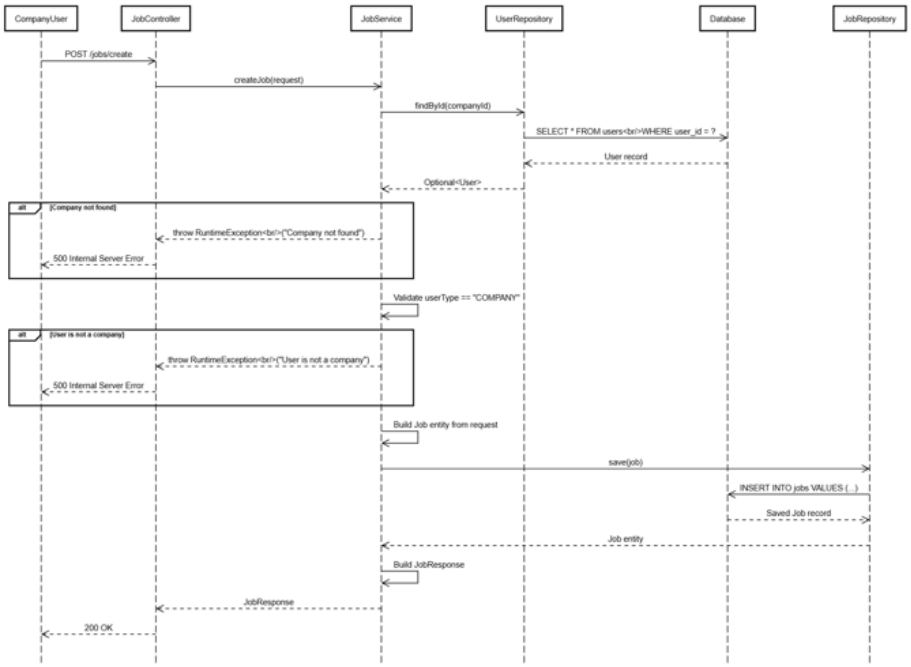


Use case Diagram for Job Posting Management

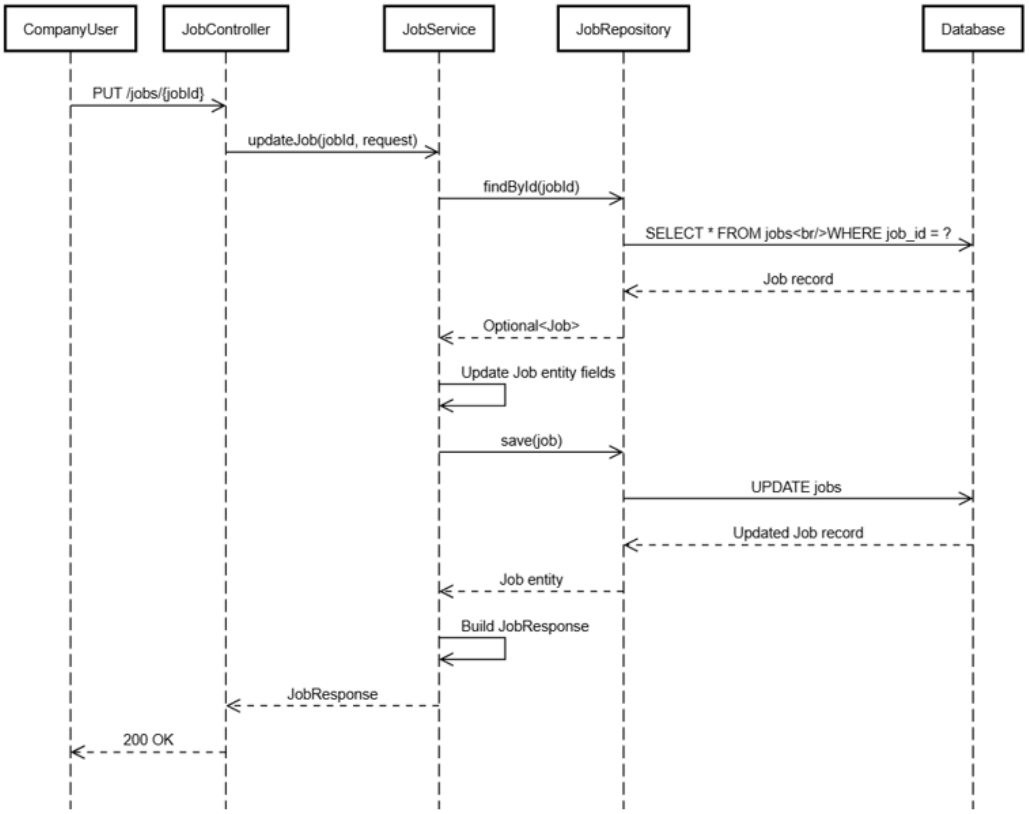


Sequence Diagrams for Job Posting

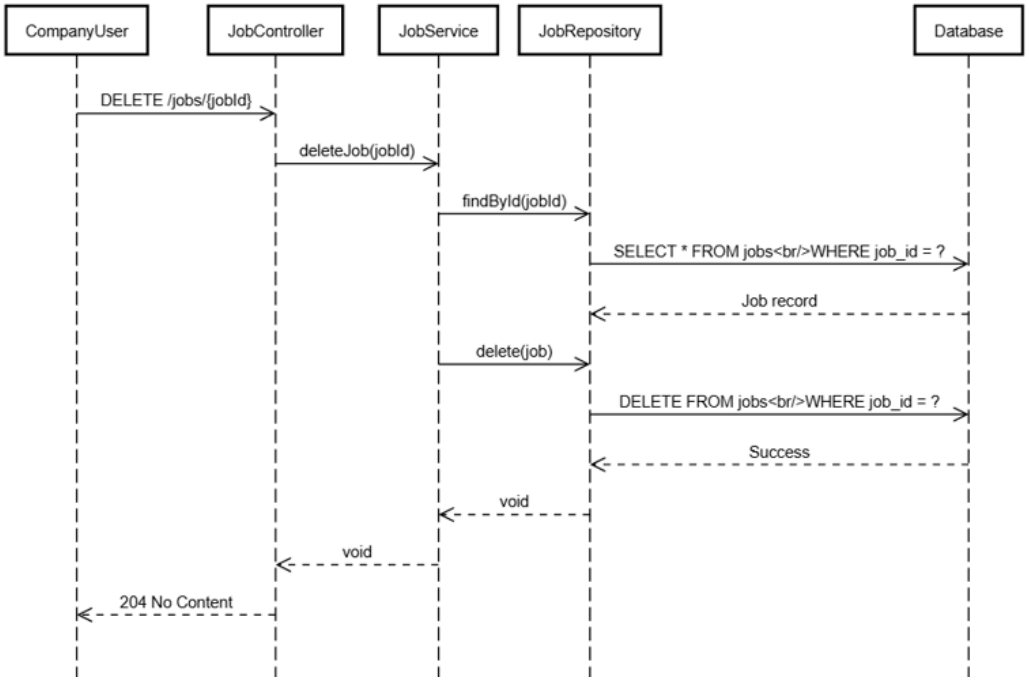
create job



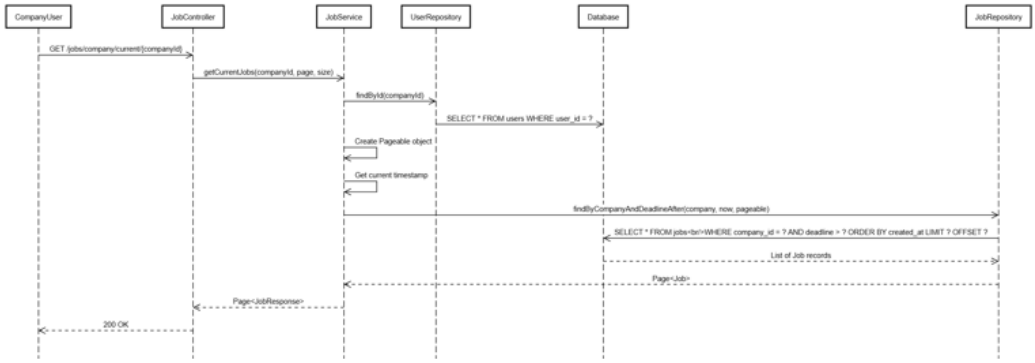
Update job



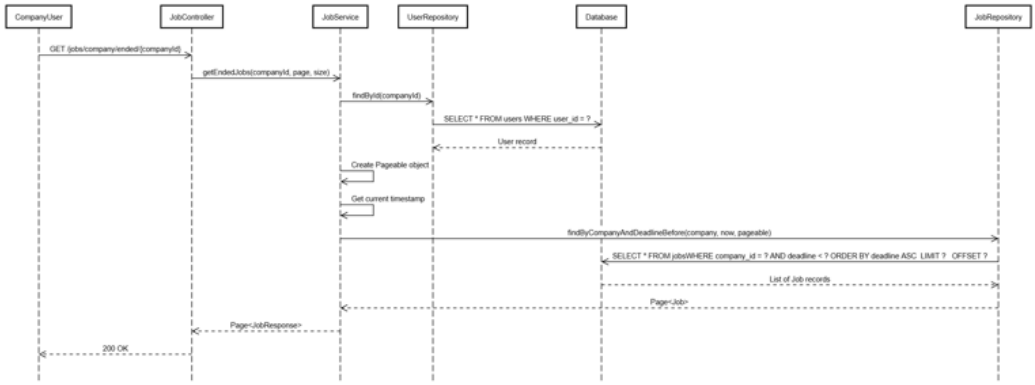
Delete job



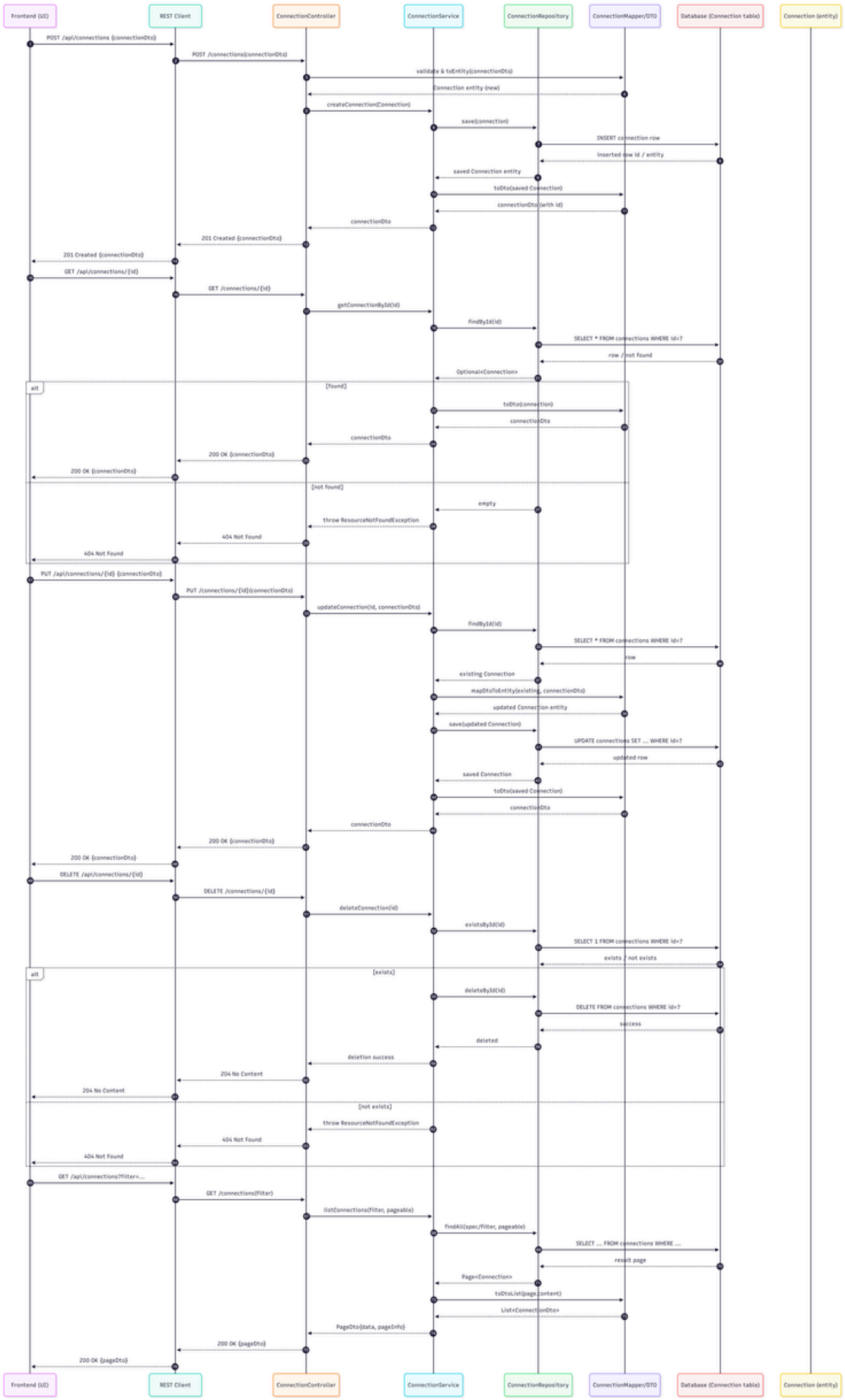
Get Current Jobs



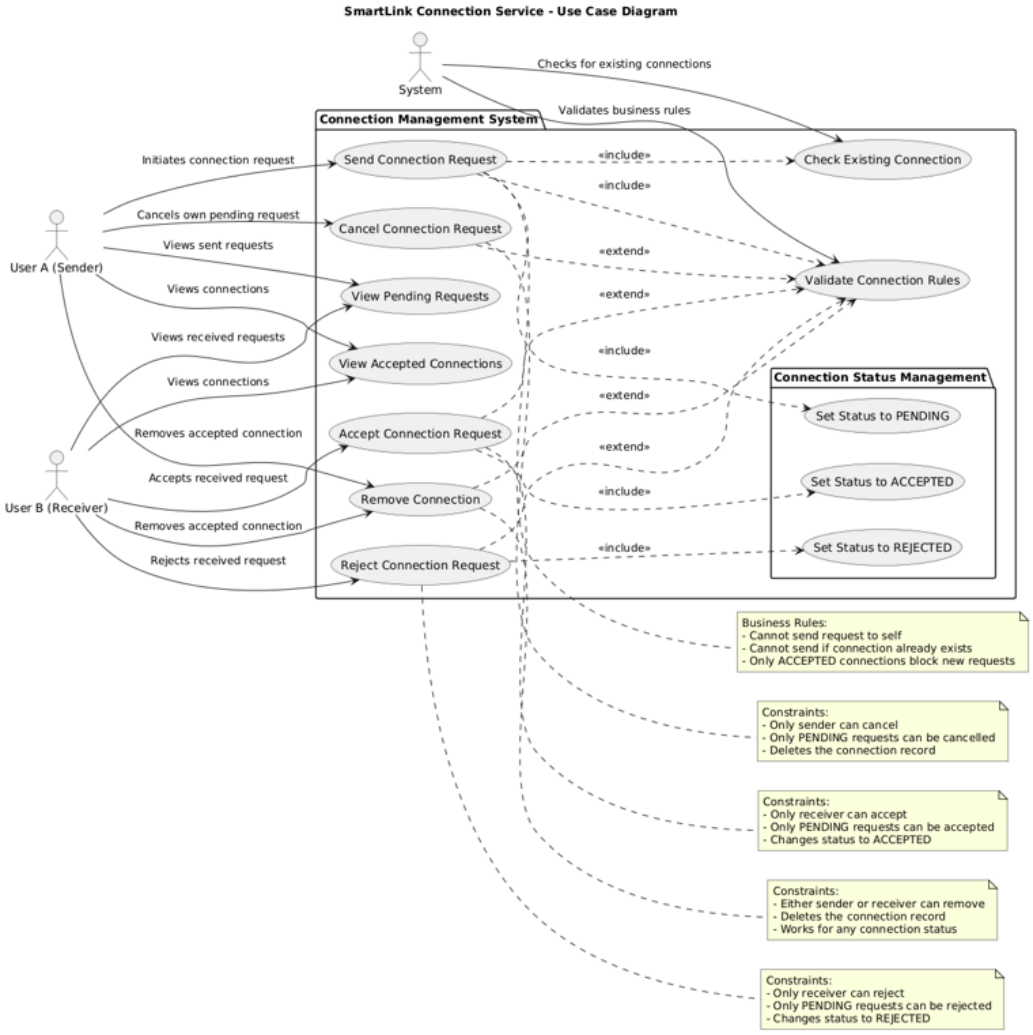
Get ended jobs



Detailed Sequence diagram for connection between users



Detailed use case for Connection service



6. System Architecture

Initial Design

- Monolithic structure
- Direct UI-to-database communication

Improved Design

- Layered Architecture:
  - Presentation Layer: React
  - Business Layer: Spring Boot Services
  - Data Layer: Repositories & MySQL

Benefits

- Better security

- Clear separation of concerns
- Improved scalability
- Easier maintenance

---

7. Workflow & Project Management

Development followed Agile methodology using Jira for task management.

---

8. Testing Strategy

Unit Testing

- Backend coverage: more than 60%
- Tools used: JUnit & Mockito
- TDD approach applied

---

9. Version Control & Collaboration

- GitHub used for source control [omar2004khaled/SmartLink: Social app with authentication and user management]
- Each member created and reviewed Pull Requests
- All PRs linked to Jira tasks for traceability [Smart-Link - Backlog - Jira]

Prepared by: SmartLink Development Team